CAPSTONE 31 — Team Git Workflow & CI Pipeline Checklist

Actionable Verification Gates for Setup, Daily Branching, Pull Requests, CI & Secret Hygiene

Phase 1: Repository Initialization & Governance Setup (Week 1)

Complete this foundational setup once at the start of the semester before any team member begins writing feature code. Grounded in GitHub Protected Branch Docs.

Status	Checklist Item	Verification Command / GitHub UI Path	Owner
[ ]	**Initialize Repository with Default `main` Branch**: Create repository with a clear `README.md` and language-specific `.gitignore` (excluding `.env`, `__pycache__/`, `.venv/`).	`git init -b main` or GitHub New Repo Wizard	Team Lead
[ ]	**Add Pull Request Template**: Commit `.github/pull_request_template.md` containing Summary, Linked Issue, Test Verification, and AI/Security Ethics checkboxes.	Verify file exists at `.github/pull_request_template.md` on `main`	Team Lead
[ ]	**Configure GitHub Actions CI Pipeline**: Commit `.github/workflows/ci.yml` running static linting (`flake8`) and automated tests (`pytest`) on `push` and `pull_request` to `main`.	Check **Actions** tab on GitHub for green workflow run	DevOps Member
[ ]	**Enable Branch Protection on `main`**: Require PR before merging, require ≥1 peer approval, dismiss stale approvals on new push, and require passing CI status checks.	`Settings → Branches → Add branch protection rule (main)`	Team Lead
[ ]	**Install Local Git Hooks on All Student Machines**: Run the hook installer so every developer's local Git enforces Conventional Commits and scans staged diffs for secrets.	`bash scripts/install_hooks.sh` (copies to `.git/hooks/commit-msg`)	All Members

Phase 2: Daily Feature Branching & Commit Hygiene (Every Task)

Follow these steps every time you pick up a task or bug fix from the team project board. Grounded in GitHub Flow and Conventional Commits v1.0.0.

Status	Checklist Item	Verification Command / Standard
[ ]	**Sync Local `main` Before Branching**: Always switch to `main` and pull the latest merged team commits before creating a new topic branch.	`git switch main && git pull --ff-only origin main`
[ ]	**Create Descriptive Topic Branch**: Name branches using `<type>/<short-kebab-description>` so teammates immediately know the scope.	`git switch -c feat/gpa-calculator` or `fix/null-credit-check`
[ ]	**Inspect Diff & Stage Intentionally**: Never run blind `git add .` without checking status and diff for temporary files, debug prints, or credentials.	`git status -sb` and `git diff` before `git add -p`
[ ]	**Write Atomic Conventional Commits**: Keep each commit focused on one logical change with a header matching `<type>(<scope>): <imperative summary>`.	`git commit -m "feat(analytics): add weighted GPA validator"`
[ ]	**Run Linter & Unit Tests Locally Before Pushing**: Catch syntax, formatting, and logic errors on your laptop in 2 seconds rather than waiting for cloud CI to fail.	`python3 scripts/lint_check.py && pytest tests/ -v`

Phase 3: Pull Request, Code Review, CI & Ethical AI Verification

Before clicking "Merge pull request", both the PR author and the peer reviewer must verify every item below. Grounded in GitHub Actions Secure Use Reference.

Status	Verification Gate	Action Required to Pass
[ ]	**Complete All PR Template Sections**	Fill out Summary, link the GitHub Issue (`Closes #N`), and check off verification boxes—never submit a blank PR body.
[ ]	**Green CI Pipeline Status Checks**	Confirm both `flake8` static analysis and `pytest` unit test matrix jobs (Python 3.11 & 3.12) show green checkmarks on the PR.
[ ]	**Zero Hardcoded Secrets or Credentials**	Verify no API keys, database URLs, passwords, or `.env` files appear in the Files Changed tab; use `${{ secrets.KEY_NAME }}` in CI.
[ ]	**AI-Generated Code & YAML Audited**	If ChatGPT, Gemini, Claude, or DeepSeek generated any code or YAML, verify action versions (@v4/@v5), permissions (`contents: read`), and edge-case tests.
[ ]	**Squash/Merge & Delete Merged Topic Branch**	After peer approval and green CI, merge the PR, delete the remote topic branch on GitHub, and run `git branch -d <branch>` locally.